TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 5: Greedy Algorithms II: Shortest Paths and Minimum Spanning Trees

Foreign Trade University

Contents

1 Introduction

Last week introduced the greedy paradigm and two proof techniques – *greedy stays ahead* and the *exchange argument* – through scheduling problems on intervals and jobs. This week we apply the same toolkit to two of the most important problems in all of algorithms, both defined on **weighted graphs**:

- the **single-source shortest path** problem, solved by *Dijkstra's algorithm* (a greedy "stays ahead" argument); and
- the **minimum spanning tree** (MST) problem, solved by *Kruskal's* and *Prim's* algorithms (both justified by a single structural fact, the *cut property*).

Both algorithms are greedy: they build a solution one vertex or one edge at a time, always taking the locally cheapest extension, and never reconsidering. As always, the implementation is short; the content is in the correctness proof. A recurring sub-theme is that Dijkstra's greedy choice is correct *only* when all edge weights are non-negative – a single negative edge can break it – which motivates the Bellman–Ford algorithm as a more robust (but slower) alternative.

Throughout, a graph $G = (V, E)$ has $n = |V|$ vertices and $m = |E|$ edges. Each edge $e = (u, v)$ carries a weight (or length, or cost) $w(e) = w(u, v) \geq 0$ unless stated otherwise. We represent graphs as *adjacency dicts*: a map from each vertex to a dict of (neighbour, weight) pairs. The shared helper file `starter_code.py` provides this representation, a seeded random weighted-graph generator, and a `UnionFind` data structure; Problem 1 asks you to build and inspect such graphs.

2 The Single-Source Shortest Path Problem

Definition 2.1 (Single-Source Shortest Paths)**.** *Given a directed or undirected graph $G = (V, E)$ with non-negative edge weights $w(e) \geq 0$ and a designated* source *vertex $s \in V$, the* single-source shortest path problem *asks, for every vertex $v \in V$, for the minimum total weight* $\mathrm{dist}(v)$ *of any path from s to v (with $\mathrm{dist}(v) = \infty$ if no path exists). The total weight of a path is the sum of the weights of its edges.*

This models a road network (weights = travel times), a communication network (weights = latencies), or any setting where we want the cheapest route from one origin to all destinations.

2.1 Dijkstra's algorithm

Dijkstra's algorithm maintains a set S of vertices whose shortest distance is already known ("finalized"), and a tentative distance $d[v]$ for every vertex. Initially $S = \emptyset$, $d[s] = 0$, and $d[v] = \infty$ for $v \neq s$. It repeatedly selects the vertex $u \notin S$ with the smallest tentative distance, adds it to S (finalizing it), and *relaxes* each outgoing edge (u, v): if $d[u] + w(u, v) < d[v]$, it lowers $d[v]$.

Algorithm 1 Dijkstra's Single-Source Shortest Paths

```
1:  d[s] ← 0;    d[v] ← ∞ for all v ≠ s
2:  S ← ∅
3:  Insert all vertices into a min-priority queue Q keyed by d[·]
4:  while Q ≠ ∅ do
5:      u ← ExtractMin(Q)                          ▷ smallest tentative distance
6:      S ← S ∪ {u}                                ▷ d[u] is now final
7:      for each edge (u, v) ∈ E with v ∉ S do
8:          if d[u] + w(u, v) < d[v] then
9:              d[v] ← d[u] + w(u, v)              ▷ relax: found a shorter route to v
10:             DecreaseKey(Q, v, d[v]);    set pred[v] ← u
11:         end if
12:     end for
13: end while
14: return d[·] (and pred[·] for path reconstruction)
```

Problem 2 asks you to implement the distance-only version using Python's `heapq`; Problem 3 adds a pred[·] array and reconstructs the actual shortest path to a target by following predecessors backward.

In practice, with a binary heap, the cleanest implementation is the *lazy* one: instead of DecreaseKey, push a new $(d[v], v)$ pair onto the heap whenever $d[v]$ improves, and discard a popped pair if its vertex has already been finalized. This is exactly what the exercise solutions do.

2.2 Correctness

Theorem 2.2 (Correctness of Dijkstra)**.** *If all edge weights are non-negative, then when Dijkstra's algorithm adds a vertex u to S, the tentative distance $d[u]$ equals the true shortest-path distance* $\mathrm{dist}(u)$ *from s to u.*

Proof (greedy stays ahead, by induction on $|S|$). We prove the invariant: for every vertex u at the moment it is added to S, $d[u] = \mathrm{dist}(u)$, and moreover $d[u]$ is the length of a genuine s-u path. The base case is $u = s$: $d[s] = 0 = \mathrm{dist}(s)$.

Inductive step. Suppose the invariant holds for all vertices currently in S, and let $u \notin S$ be the next vertex extracted (so $d[u] \le d[v]$ for all $v \notin S$). Because every relaxation sets $d[v]$ to the length of an actual path, $d[u] \ge \mathrm{dist}(u)$ always; we must show $d[u] \le \mathrm{dist}(u)$, i.e. no path is shorter than $d[u]$.

Consider any s-u path P. Since $s \in S$ and $u \notin S$, P must leave S at some point: let (x, y) be the first edge of P with $x \in S$ and $y \notin S$. By the inductive hypothesis $d[x] = \mathrm{dist}(x)$, and when x was added to S the edge (x, y) was relaxed, so $d[y] \le d[x] + w(x, y)$. The length of P is at least the length of its prefix up to y, which is at least $\mathrm{dist}(x) + w(x, y) = d[x] + w(x, y) \ge d[y]$. Since $y \notin S$ and u was chosen as the minimum, $d[u] \le d[y]$. Putting it together, the length of P is at least $d[y] \ge d[u]$. As P was arbitrary, $\mathrm{dist}(u) \ge d[u]$. Hence $d[u] = \mathrm{dist}(u)$. □

The single place where non-negativity is used is the step "the length of P is at least the length of its prefix up to y": this needs every edge *after* y on P to add a non-negative amount. With a negative edge this can fail, and the proof – and the algorithm – break (see §2.4).

2.3 Running time

The cost is dominated by the priority-queue operations: n ExtractMin operations and up to m DecreaseKey (or push) operations. With a **binary heap**, each costs $O(\log n)$, giving

$$O\big((m + n) \log n\big),$$

which is $O(m \log n)$ for a connected graph (where $m \ge n - 1$). With a more sophisticated Fibonacci heap, DecreaseKey is $O(1)$ amortized and the bound improves to $O(m + n \log n)$, but the binary-heap version is what is used in practice and in this course.

2.4 Why Dijkstra fails on negative edges

Dijkstra's greedy choice – "finalize the nearest unfinished vertex, and never touch it again" – is justified only because, with non-negative weights, extending a path never decreases its length. A negative edge violates this: a seemingly-distant vertex might be reachable cheaply via a detour that ends with a negative edge, but by then Dijkstra has already finalized the relevant vertex.

Example 2.3 (A negative edge breaks Dijkstra)**.** *Take vertices s, a, b with directed edges $s \to a$ of weight 2, $s \to b$ of weight 3, and $b \to a$ of weight -2. The true shortest distance to a is via $s \to b \to a$: $3 + (-2) = 1$. But Dijkstra extracts a first (tentative distance $2 < 3$), finalizes* $\mathrm{dist}(a) = 2$*, and never revisits it – so it reports the wrong answer 2. Problem 6 asks you to construct exactly this counter-example and confirm that Dijkstra and Bellman–Ford disagree on it.*

Algorithm 2 Bellman–Ford Single-Source Shortest Paths

```
d[s] ← 0;   d[v] ← ∞ for all v ≠ s
for i = 1 to n − 1 do
    for each edge (u, v) ∈ E do
        if d[u] + w(u, v) < d[v] then
            d[v] ← d[u] + w(u, v)                          ▷ relax
        end if
    end for
end for
for each edge (u, v) ∈ E do                  ▷ one extra pass detects a negative cycle
    if d[u] + w(u, v) < d[v] then
        return "negative cycle reachable"
    end if
end for
return d[·]
```

3 The Bellman–Ford Algorithm

When edges may be negative, we need a different approach. The Bellman–Ford algorithm relaxes *every* edge, $n-1$ times in a row.

Theorem 3.1 (Correctness of Bellman–Ford)**.** *If G has no negative cycle reachable from s, then after the $n-1$ rounds of relaxation, $d[v] = \mathrm{dist}(v)$ for every v. Moreover, a further relaxation is possible on some edge* iff *a negative cycle is reachable from s.*

Proof sketch. A shortest path that uses no negative cycle has at most $n-1$ edges. By induction on i, after i rounds $d[v]$ is at most the weight of the shortest s-v path using at most i edges (relaxing every edge each round can only propagate the best "i-edge" distance one more hop). After $n-1$ rounds every shortest path (at most $n-1$ edges) has been captured. If, in the extra pass, some edge can still be relaxed, then there is a closed walk on which the total weight strictly decreases – i.e. a negative cycle; conversely a negative cycle always admits such a relaxation. □

Each round relaxes all m edges, so Bellman–Ford runs in $O(nm)$ time – slower than Dijkstra's $O((m+n)\log n)$, the price paid for handling negative edges and detecting negative cycles. Problem 4 asks you to implement Bellman–Ford returning `None` on a negative cycle; Problem 5 verifies that Dijkstra and Bellman–Ford agree on random *non-negative* graphs.

4 The Minimum Spanning Tree Problem

Definition 4.1 (Minimum Spanning Tree)**.** *Given a connected, undirected graph $G = (V, E)$ with edge weights $w(e)$, a* spanning tree *is a subset $T \subseteq E$ of $n-1$ edges that connects all n vertices and contains no cycle. A* minimum spanning tree *(MST) is a spanning tree of minimum total weight $\sum_{e \in T} w(e)$.*

An MST is the cheapest way to wire up a network so that every node can reach every other: think of laying cable to connect cities, or a chip's clock- distribution network. Two classic greedy algorithms solve it – Kruskal's and Prim's – and both rest on a single structural fact, the cut property.

4.1 The cut and cycle properties

Definition 4.2 (Cut)**.** *A* cut *of G is a partition of V into two non-empty sets $(S, V \setminus S)$. An edge* crosses *the cut if it has exactly one endpoint in S.*

To keep the statements clean, we first assume all edge weights are *distinct* (ties can be broken consistently, e.g. by edge index; with that convention the MST is unique and every claim below goes through).

Theorem 4.3 (Cut Property)**.** *Let $(S, V \setminus S)$ be any cut, and let e be the* minimum-weight *edge crossing it. Then e belongs to every minimum spanning tree of G.*

Proof (exchange argument). Let $e = (u, v)$ with $u \in S$, $v \notin S$, and suppose for contradiction that some MST T does not contain e. Adding e to T creates exactly one cycle C (since T is a spanning tree, $T \cup \{e\}$ has one cycle, which passes through e). The cycle C must cross the cut an even number of times, so it contains at least one other crossing edge $e' \neq e$. Since e is the minimum-weight crossing edge and weights are distinct, $w(e) < w(e')$. Now form $T' = (T \setminus \{e'\}) \cup \{e\}$: removing e' from the cycle keeps the graph connected and acyclic, so T' is a spanning tree, and $w(T') = w(T) - w(e') + w(e) < w(T)$. This contradicts the minimality of T. Hence every MST contains e. □

Theorem 4.4 (Cycle Property)**.** *Let C be any cycle in G, and let f be the* maximum-weight *edge on C. Then f belongs to* no *minimum spanning tree of G.*

Proof. Suppose some MST T contains $f = (x, y)$. Removing f splits T into two components, with x on one side (call its vertex set S) and y on the other. Edge f crosses the cut $(S, V \setminus S)$. The cycle C also crosses this cut, so besides f it contains another crossing edge f' with (since f is the unique heaviest on C) $w(f') < w(f)$. Replacing f by f' reconnects the two components into a spanning tree T' with $w(T') < w(T)$, contradicting minimality. □

The cut property tells us which edges *must* be in the MST; the cycle property tells us which edges *cannot* be. Problem 10 asks you to verify the cut property empirically, and Problem 11 uses both properties to classify a given edge as belonging to *every*, *some*, or *no* MST.

4.2 Kruskal's algorithm

Kruskal's algorithm sorts the edges by weight and adds each one that does not create a cycle – detected with a union-find (disjoint-set) data structure.

Algorithm 3 Kruskal's Minimum Spanning Tree

```
Sort edges so that w(e_1) ≤ w(e_2) ≤ ··· ≤ w(e_m)
T ← ∅;   initialize union-find with each vertex its own set
for i = 1 to m do
    let e_i = (u, v)
    if FIND(u) ≠ FIND(v) then                    ▷ u, v in different components
        T ← T ∪ {e_i};   UNION(u, v)
    end if
end for
return T
```

Theorem 4.5. *Kruskal's algorithm produces a minimum spanning tree.*

Proof. Every edge $e = (u, v)$ that Kruskal *adds* is the minimum-weight edge crossing the cut $(S, V \setminus S)$, where S is the set of vertices in u's current component: e connects two different components, and every cheaper edge was already processed and either added (staying inside a component) or discarded – in particular, no cheaper edge crosses this cut, since had one existed it would have merged the components earlier. By the cut property (Theorem 4.3) e belongs to every MST, so every edge Kruskal adds is safe. Since Kruskal adds edges until all vertices are connected (it never creates a cycle), it returns a spanning tree, all of whose edges belong to the MST – hence the MST itself. □

Union-find and running time. The bottleneck is sorting the edges: $O(m \log m) = O(m \log n)$. The union-find operations – with *path compression* and *union by rank* – cost nearly $O(1)$ amortized (formally, $O(\alpha(n))$ where α is the inverse Ackermann function, effectively constant), so the total is $O(m \log n)$. The `UnionFind` class in `starter_code.py` implements both optimizations; Problem 7 asks you to build Kruskal's algorithm on top of it and return both the MST edge set and its total weight.

4.3 Prim's algorithm

Prim's algorithm grows a single tree outward from an arbitrary start vertex, always adding the cheapest edge that leaves the current tree – the direct graph analogue of Dijkstra, using a heap of candidate crossing edges.

Algorithm 4 Prim's Minimum Spanning Tree

```
Pick any start vertex r;    S ← {r};    T ← ∅
Push every edge incident to r onto a min-heap Q, keyed by weight
while |S| < n and Q ≠ ∅ do
    (w, v) ← ExtractMin(Q)                          ▷ cheapest candidate edge
    if v ∈ S then continue                          ▷ stale entry; skip
    end if
    S ← S ∪ {v};    add the chosen edge to T
    for each edge (v, x) with x ∉ S do
        Push (w(v, x), x) onto Q
    end for
end while
return T
```

Theorem 4.6. *Prim's algorithm produces a minimum spanning tree.*

Proof. At every step, S is the set of vertices in the current tree, and Prim adds the minimum-weight edge crossing the cut $(S, V \setminus S)$. By the cut property (Theorem 4.3) this edge belongs to every MST, so every edge added is safe. After $n - 1$ such additions, $S = V$ and T is a spanning tree composed entirely of MST edges – the MST. □

Using a binary heap, Prim's algorithm runs in $O((m + n) \log n)$, the same as Dijkstra. Problem 8 asks you to implement it (returning the total MST weight), and Problem 9 verifies that Kruskal and Prim always agree on the total weight for random connected graphs – a useful empirical check, since (with ties) they may pick different edge sets but must achieve the same minimum total.

5 Application: Single-Link Clustering

A pleasant payoff of the MST machinery is an optimal clustering algorithm. Suppose we have n points (objects) with a pairwise distance between each pair, and we want to partition them into k groups (*clusters*). A natural objective is to *maximize the spacing*: the *spacing* of a clustering is the minimum distance between any two points that lie in *different* clusters, and we want this minimum gap to be as large as possible.

Definition 5.1 (k-Clustering of Maximum Spacing)**.** *Given a complete weighted graph on the points (weights = distances) and an integer k, partition the vertices into k non-empty clusters so that the minimum weight of an edge joining two different clusters (the* spacing*) is maximized.*

Theorem 5.2 (Single-link clustering via Kruskal)**.** *The following greedy procedure produces a k-clustering of maximum spacing: run Kruskal's algorithm on the points, but* stop *as soon as the union-find has exactly k components. Equivalently, build the MST and delete its $k-1$ most-expensive edges; the remaining k components are the clusters, and the spacing equals the weight of the $(k-1)$-th most expensive MST edge (the next edge Kruskal would have added).*

Proof sketch. Running Kruskal until k components remain is exactly adding the $n-k$ cheapest "component-merging" edges; the spacing of the resulting clustering is the weight d^* of the next edge Kruskal would add. Consider any other k-clustering $\mathcal{C}'$ that differs from ours. Then two points p, q in the same cluster of ours lie in different clusters of $\mathcal{C}'$. The Kruskal path joining p and q uses only edges of weight $\leq d^*$, and somewhere along it two consecutive points are split by $\mathcal{C}'$ – giving $\mathcal{C}'$ a crossing edge of weight $\leq d^*$, so its spacing is at most d^*. Hence no clustering beats ours. □

Problem 12 asks you to implement this: run Kruskal until k components remain and return both the clustering and the spacing.

6 Summary and Looking Ahead

This week applied greedy design to two cornerstone graph problems:

- **Shortest paths.** *Dijkstra's algorithm* greedily finalizes the nearest unprocessed vertex; the "greedy stays ahead" proof (Theorem 2.2) works only for non-negative weights, in $O((m+n)\log n)$ with a binary heap. A single negative edge breaks it, motivating *Bellman–Ford* ($O(nm)$), which also detects negative cycles.
- **Minimum spanning trees.** The *cut property* and *cycle property* (Theorems 4.3, 4.4) determine exactly which edges must, and must not, appear in an MST. Both *Kruskal's* algorithm (sort + union-find) and *Prim's* algorithm (grow + heap) are correct because every edge they add is a minimum crossing edge of some cut.
- **Application.** Single-link k-clustering is just Kruskal stopped early, and it provably maximizes the spacing.

The unifying lesson is that one structural fact (the cut property) justifies two different-looking greedy algorithms, just as one proof template (greedy stays ahead) justified Dijkstra. Next week we change paradigms entirely: from greedy to **divide and conquer** (Kleinberg & Tardos Ch. 5), where we solve a problem by splitting it into independent subproblems, solving each recursively, and combining – with the running time governed by recurrence relations and the Master Theorem.